

Lab 4: Symmetric Key and Hashing

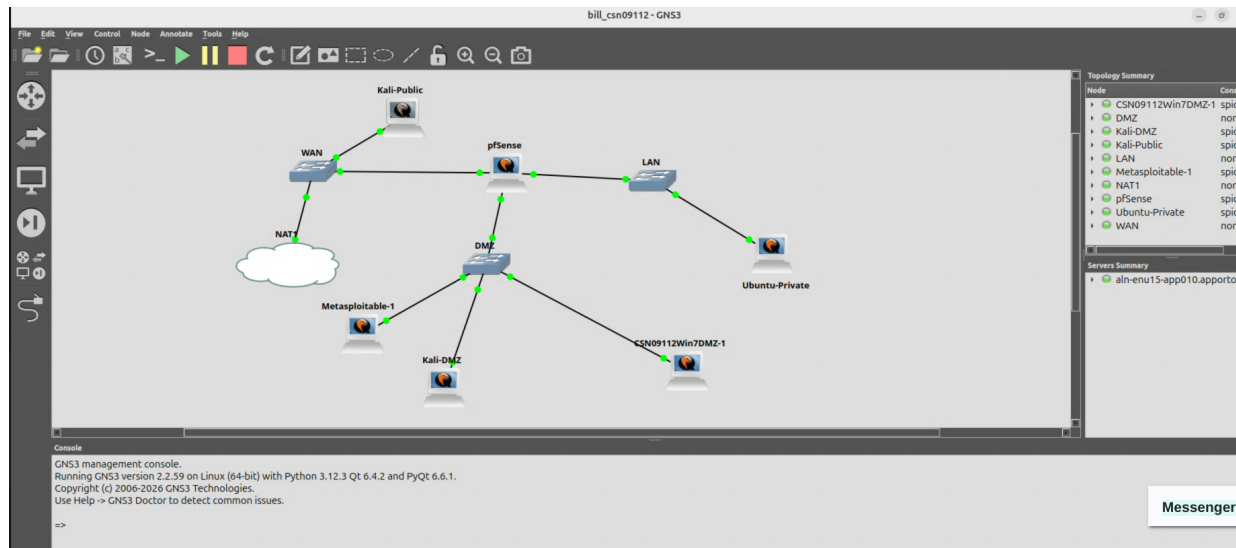
Demo: <http://youtu.be/3n2TMpHqE18>

2 Lab 4: Symmetric Key and Hashing

Part 1 Demo: here

We will use OpenSSL for a few tutorial examples. If you want to find out more about the program, discover here.

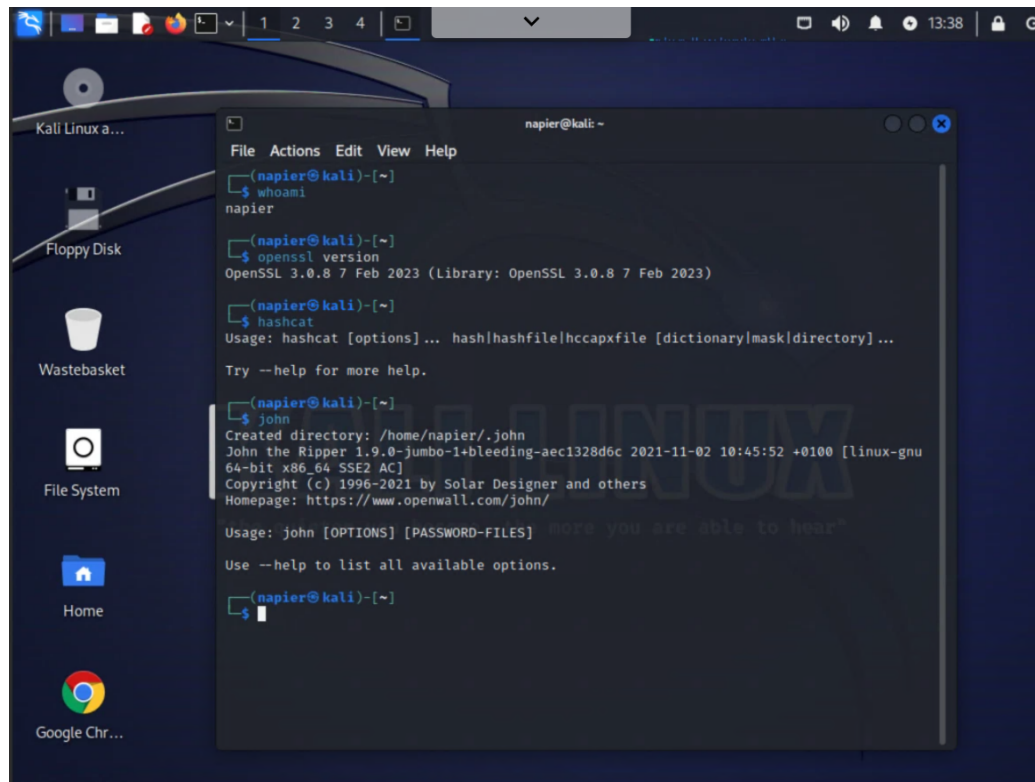
In the Virtual Labs (as used in Labs 1 & 2), open GNS3, and start and run the Kali-Public instance



Username: napier / password: napier123

You may find it useful to open this repo with chrome within the instance or easy copy/paste of the instructions.

Kali comes with all the tools you need for this lab.



Optionally if you plan on continuing your journey in the world of cybersecurity, you may want to consider installing your own virtual machines with VirtualBox or VMWare (Windows/Linux) or UTM (Mac). Instructions for this can be found in the Appendix and the demonstrators will be happy to help in the JKCC.

A Symmetric Key

No	Description	Result
1	Get a terminal open on Linux.	
2	Use: <code>openssl list -cipher-commands</code>	Outline five encryption methods that are supported:
3	Use: <code>openssl version</code>	Outline the version of OpenSSL:
4	Using openssl and the command in the form: <code>openssl prime -hex 1111</code>	Check if the following are prime numbers:
5	Now create a file named myfile.txt (either use nano or another editor). Next. encrypt with aes-256-cbc <code>openssl enc -aes-256-cbc -in myfile.txt -out encrypted.bin -pbkdf2</code> and enter your password.	Use the following command to view the output file: <code>cat encrypted.bin</code> Is it easy to write out or transmit the output: [Yes][No]. What does the -pbkdf2 part do?
6	Now repeat the previous command and add the -base64 option. <code>openssl enc -aes-256-cbc -in myfile.txt -out encrypted.bin -base64 -pbkdf2</code>	Use the following command to view the output file: <code>cat encrypted.bin</code> Is it easy to write out or transmit the output: [Yes][No]
7	Now repeat the previous command and observe the encrypted output. <code>openssl enc -aes-256-cbc -in myfile.txt -out encrypted.bin -base64 -pbkdf2</code>	Has the output changed? [Yes][No] Why has it changed?
8	Now let's decrypt the encrypted file with the correct format: <code>openssl enc -d -aes-256-cbc -in encrypted.bin -pass pass:napier -base64 -pbkdf2</code> Has the output been decrypted correctly?	What happens when you use the wrong password?
9	If you are working in the lab, now give your secret passphrase to your neighbour, and get them to encrypt a secret message for you. To receive a file, you listen on a given port (such as Port 1234) <code>nc -l -p 1234 > enc.bin</code> And then send to a given IP address with: <code>nc -w 3 [IP] 1234 < enc.bin</code>	Did you manage to decrypt their message? [Yes][No]

10. With OpenSSL, we can define a fixed salt value that has been used in the cipher process. For example, in Linux:

```
echo -n "Hello" | openssl enc -aes-128-cbc -pass pass:"london" -e -base64 -S 241fa86763b85341 -pbkdf2
```

and then decrypt:

```
echo 9Z+NtmCdQSpmRl+eZebFXQ== | openssl enc -aes-128-cbc -pass pass:"london" -d -base64 -S 241fa86763b85341 -pbkdf2  
Hello
```

For a ciphertext for 256-bit AES CBC and a message of “Hello” with a salt value of 241fa86763b85341, try the following passwords, and determine the password used for a ciphertext of tZCdiQE4L6QT+Dff82F5bw== [qwerty][inkwell][london][paris][cake]

11. Now, use the decryption method to prove that you can decrypt the ciphertext.

```
echo tZCdiQE4L6QT+Dff82F5bw== | openssl enc -aes-256-cbc -pass pass:"password" -d -base64 -S 241fa86763b85341 -pbkdf2
```

Did you confirm the right password? [Yes/No]

12. Investigate the following commands by running them several times:

```
echo -n "Hello" | openssl enc -aes-128-cbc -pass pass:"london" -e -base64 -S 241fa86763b85341 -pbkdf2  
echo -n "Hello" | openssl enc -aes-128-cbc -pass pass:"london" -e -base64 -salt -pbkdf2
```

What do you observe? Why do you think causes the changes?

13. We don't always need to use a file to save the cipher, too. With the following, we will encrypt the plaintext of "melon":

```
echo "melon" | openssl enc -e -aes-128-cbc -pass pass:stirling -base64 -pbkdf2  
U2FsdGVkX18cryB3vdNj+Tax1PGecO6ZOW2WL1LmdKQ=
```

and then we can decrypt with:

```
echo "U2FsdGVkX18cryB3vdNj+Tax1PGecO6ZOW2WL1LmdKQ=" | openssl enc -d -aes-128-cbc -pass pass:stirling -base64 -pbkdf2  
  
melon
```

Now crack the following cipher using a Scottish city as a password (the password is in lower case):

U2FsdGVkX1+7VpBGwevibQGgescaz5nsArtGLNqFaXk=

What is the fruit in the plaintext?

Now try:

U2FsdGVkX18vpjgccu7VkpZrkncqADuy1kVKU9LbLec=

What is the fruit?

B Hashing

Video: [here](#)

Q1

Using: [here](#) Match the hash signatures with their words (“Falkirk”, “Edinburgh”, “Glasgow” and “Stirling”).

03CF54D8CE19777B12732B8C50B3B66F

Is it [Falkirk][Edinburgh][Glasgow][Stirling]?

D586293D554981ED611AB7B01316D2D5

Is it [Falkirk][Edinburgh][Glasgow][Stirling]?

48E935332AADEC763F2C82CDB4601A25

Is it [Falkirk][Edinburgh][Glasgow][Stirling]?

EE19033300A54DF2FA41DB9881B4B723

Is it [Falkirk][Edinburgh][Glasgow][Stirling]?

Q2

Using: here, determine the number of hex characters in the following hash signatures.

MD5 hex chars:

SHA-1 hex chars:

SHA-256 hex chars:

How does the number of hex characters relate to the length of the hash signature: |

Q3

The hashes below have been found in a /etc/shadow file, determine the matching password (the passwords are password, napier, inkwell and Ankle123) - Remember to change the salt!

To find the password, we determine the salt value, and try each password. For example the salt value for
bill:\$apr1\$waZS/8Tm\$jDZmiZBct/c2hysERcZ3m1 is waZS/8Tm. To check the password and salt, we can run:

```
openssl passwd -apr1 -salt waZS/8Tm napier
$apr1$waZS/8Tm$jDZmiZBct/c2hysERcZ3m1
```

Now try these ones by trying each of the possible passwords:

```
bill:$apr1$waZS/8Tm$jDZmiZBct/c2hysERcZ3m1
```

Bill's password:

```
mike:$apr1$mKfrJquI$Kx0CL9krmqhCu0SHKqp5Q0
```

Mike's password:

```
fred:$apr1$Jbe/hCIb$/k3A4kjpJyC06BUUaPRKs0
```

Fred's password:

```
ian:$apr1$0GyPhsLi$jTTzW0HNS4Cl5ZEoyFLjB.
```

Ian's password:

```
jane: $1$rqOIRBBN$R2pOQH9egTTVN1N1st2U7.
```

Jane's password:

[Hint: openssl passwd -apr1 -salt ZaZS/8TF napier]

Q4

First you need to get the files

```
curl -LO https://raw.githubusercontent.com/billbuchanan/csn09112/master/week05_secretkey/labs/files02.zip
unzip files02.zip -d files02
cd files02
```

the files should have the following MD5 hashes :

```
MD5(1.txt)= 5d41402abc4b2a76b9719d911017c592
MD5(2.txt)= 69faab6268350295550de7d587bc323d
MD5(3.txt)= fea0f1f6fede90bd0a925b4194deac11
MD5(4.txt)= d89b56f81cd7b82856231e662429bcf2
```

Which file(s) have been modified:

Note: Use can use md5sum to compute MD5 hashes.


```
md5sum 1.txt
```

Q5

First you need to get the files

```
curl -LO https://raw.githubusercontent.com/billbuchanan/csn09112/master/week05_secretkey/labs/letters.zip
unzip letters.zip -d letters
cd letters
```

View the letters. Are they different? Now determine the MD5 signature for them. What can you observe from the result?

C Hashing Cracking (MD5)

Video: [here](#)

The current Hashcat version on Kali has problems with a lack of memory. To overcome this, install Hashcat 6.0.0. On Kali on your public network, first download Hashcat 6.0.0:

```
wget https://hashcat.net/files/hashcat-6.0.0.7z
```

Next, unzip it into your home folder:

```
p7zip -d hashcat-6.0.0.7z
```

Then from your home folder, set up a link to Hashcat 6.0.0:

```
ln -s hashcat-6.0.0/hashcat.bin hashcat
```

Then the version should show Verion 6.0.0:

```
# ./hashcat --version
v6.0.0
```

Q1

Next create a word file (words) with the words of “napier”, “password” “Ankle123” and “inkwell”

```
nano words
```

Then create a word file (hash1) with the following hashes

```
232DD5D7274E0D662F36C575A3BD634C
5F4DCC3B5AA765D61D8327DEB882CF99
6D5875265D1979BDAD1C8A8F383C5FF5
04013F78ACCFEC9B673005FC6F20698D
```

Using hashcat crack the following MD5 signatures (hash1):

Command used:

```
./hashcat -m 0 hash1 words
```

232DD...634C Is it [napier][password][Ankle123][inkwell]?

5F4DC...CF99 Is it [napier][password][Ankle123][inkwell]?

6D587...5FF5 Is it [napier][password][Ankle123][inkwell]?

04013...698D Is it [napier][password][Ankle123][inkwell]?

Note: use the --show option to show the results of the cracking.

Q3

Using the method used in the Q2 part of this tutorial, find the hashes of the following for names of fruits such as "orange", "apple", "banana", "pear", "peach" (the fruits are all in lowercase):

```
FE01D67A002DFA0F3AC084298142ECCD
1F3870BE274F6C49B3E31A0C6728957F
72B302BF297A228A75730123EFEF7C41
8893DC16B1B2534BAB7B03727145A2BB
889560D93572D538078CE1578567B91A
```

FE01D:

1F387:

72B30:

8893D:

88956:

Hashing Cracking (LM Hash/Windows)

All of the passwords in this section are in lowercase. <http://youtu.be/Xvbk2nSzEPk>

Q1

```
john --format=NT --wordlist=words hash.txt
# words is the list of different passwords you need to create/update
# hash.txt is the hashed password you need to create by copy/pasting the hashes below
```

Using John the Ripper, and using a word list with the names of fruits, crack the following pwdump passwords:

```
fred:500:E79E56A8E5C6F8FEAAD3B435B51404EE:5EBE7DFA074DA8EE8AEF1FAA2BBDE876:::
```

Fred's password:

```
bert:501:10EAF413723CBB15AAD3B435B51404EE:CA8E025E9893E8CE3D2CBF847FC56814:::
```

Bert's password:

Q2

Using John the Ripper, the following pwdump passwords (they are names of major Scottish cities/towns):

```
Admin:500:629E2BA1C0338CE0AAD3B435B51404EE:9408CB400B20ABA3DFEC054D2B6EE5A1:::  
fred:501:33E58ABB4D723E5EE72C57EF50F76A05:4DFC4E7AA65D71FD4E06D061871C05F2:::  
bert:502:BC2B6A869601E4D9AAD3B435B51404EE:2D8947D98F0B09A88DC9FCD6E546A711:::
```

Admin:

Fred:

Bert:

Q3

On Kali, and using John the Ripper, crack the following pwdump passwords (they are the names of animals):

```
fred:500:5A8BB08EFF0D416AAD3B435B51404EE:85A2ED1CA59D0479B1E3406972AB1928:::  
bert:501:C6E4266FEBEBD6A8AAD3B435B51404EE:0B9957E8BED733E0350C703AC1CDA822:::  
admin:502:333CB006680FAF0A417EAF50CFAC29C3:D2EDBC29463C40E76297119421D2A707:::
```

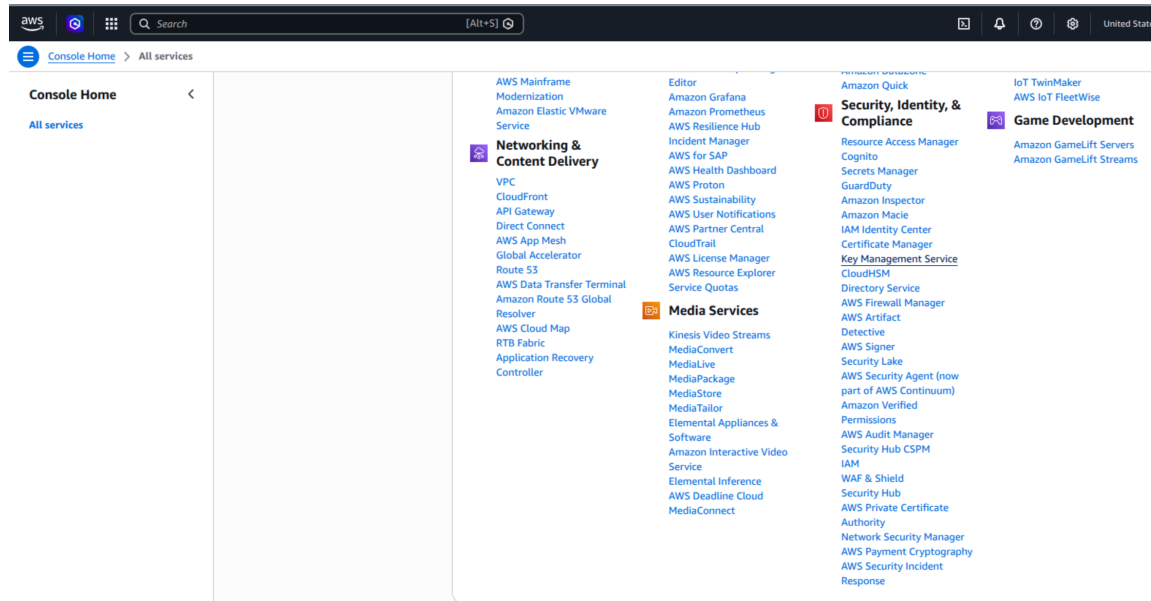
Fred:

Bert:

Admin:

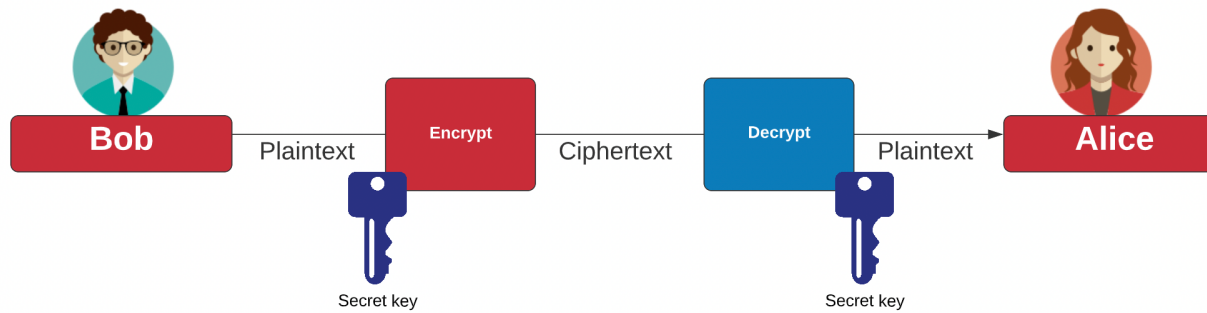
D AWS Cryptography

We are generally moving our security into the public cloud, and thus, many of our keys are stored there. In AWS, we use KMS (Key Management System), and can create either symmetric keys or asymmetric keys (public keys). In the services dashboard in your AWS learner's lab, select Key Management Service:

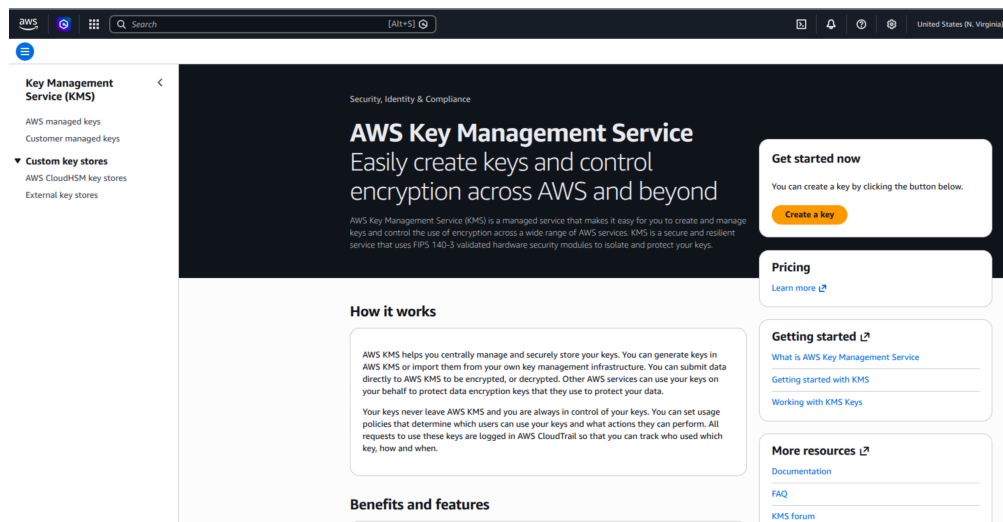


Symmetric key

With symmetric key encryption, Bob and Alice use the same encryption key to encrypt and decrypt:



Normally, we use AES encryption for this. Initially, in KMS, we create a new key within our Customer-managed keys:



and then create the key:

Introducing the new Create key experience
We've improved the create key experience with an enhanced policy editor. Let us know what you think or you can use the old experience.

Step 1: Configure key

- Step 2: Add labels
- Step 3 - optional: Define key administrative permissions
- Step 4 - optional: Define key usage permissions
- Step 5 - optional: Edit key policy
- Step 6: Review

Configure key

Key type [Help me choose](#)

☒ **Symmetric**
A single key used for encrypting and decrypting data or generating and verifying HMAC codes

☐ **Asymmetric**
A public and private key pair used for encrypting and decrypting data, signing and verifying messages, or deriving shared secrets

Key usage [Help me choose](#)

☒ **Encrypt and decrypt**
Use the key only to encrypt and decrypt data.

☐ **Generate and verify MAC**
Use the key only to generate and verify hash-based message authentication codes (HMAC).

Advanced options

[Cancel](#) [Next](#)

Next, we give it a name:

Introducing the new Create key experience
We've improved the create key experience with an enhanced policy editor. Let us know what you think or you can use the old experience.

Step 2: Add labels

- Step 1: Configure key
- Step 3 - optional: Define key administrative permissions
- Step 4 - optional: Define key usage permissions
- Step 5 - optional: Edit key policy
- Step 6: Review

Add labels

Alias
You can change the alias at any time. [Learn more](#)

Alias
MySymKey

Description - optional
You can change the description at any time.

Description
MySymKey for Bill's lab

Tags - optional
You can use tags to categorize and identify your KMS keys and help you track your AWS costs. When you add tags to AWS resources, AWS generates a cost allocation report for each tag. [Learn more](#)
This key has no tags.

[Add tag](#)
You can add up to 50 more tags.

And then define the administrative permission (those who can delete it):

The screenshot shows the 'Define key administrative permissions - optional' step in the AWS IAM console. A sidebar on the left lists the steps: Step 1: Configure key, Step 2: Add labels, Step 3: optional: Define key administrative permissions (highlighted), Step 4: optional: Define key usage permissions, Step 5: optional: Edit key policy, and Step 6: Review. The main content area has a title 'Define key administrative permissions - optional' and a sub-header 'Key administrators (1/24)'. Below this, a text block explains that IAM users and roles are authorized to manage the key via the KMS API. A search bar 'Search Key administrators' is present. A table lists four roles: 'RoleForLambdaModLabRole', 'vocareum', 'vocareum-eventbridge', and 'voclabs'. The 'voclabs' role is selected with a checkbox. Below the table, a section titled 'Key deletion' has a checkbox 'Allow key administrators to delete this key' which is also checked. At the bottom are buttons for 'Cancel', 'Skip to Review', 'Previous', and 'Next'.

☒	Name	Path	Type
☐	RoleForLambdaModLabRole	/	Role
☐	vocareum	/	Role
☐	vocareum-eventbridge	/	Role
☒	voclabs	/	Role

And the usage:

The screenshot shows the 'Define key usage permissions - optional' step in the AWS IAM console. The sidebar on the left is the same as the previous screenshot, but 'Define key usage permissions' is now highlighted. The main content area has a title 'Define key usage permissions - optional' and a sub-header 'Key users (1/24)'. Below this, a text block explains that IAM users and roles are authorized to use the key in cryptographic operations. A search bar 'Search Key users' is present. A table lists the same four roles as the previous screenshot: 'RoleForLambdaModLabRole', 'vocareum', 'vocareum-eventbridge', and 'voclabs'. The 'voclabs' role is selected with a checkbox. Below the table, a section titled 'Other AWS accounts' has a text block explaining that administrators of specified accounts are responsible for managing permissions. An 'Add another AWS account' button is present. At the bottom are buttons for 'Cancel', 'Skip to Review', 'Previous', and 'Next'.

☒	Name	Path	Type
☐	RoleForLambdaModLabRole	/	Role
☐	vocareum	/	Role
☐	vocareum-eventbridge	/	Role
☒	voclabs	/	Role

The policy is then:

```
{
  "Id": "key-consolepolicy-3",
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Enable IAM User Permissions",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::590269919252:root"
      },
      "Action": "kms:*",
      "Resource": "*"
    },
    {
      "Sid": "Allow access for Key Administrators",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::590269919252:role/voclabs"
      },
      "Action": [
        "kms:Create*",
        "kms:Describe*",
        "kms:Enable*",
        "kms:List*",
        "kms:Put*",
        "kms:Update*",
        "kms:Revoke*",
        "kms:Disable*",
        "kms:Get*",
        "kms>Delete*",
        "kms:TagResource",
        "kms:UntagResource",
        "kms:ScheduleKeyDeletion",
        "kms:CancelKeyDeletion",
        "kms:RotateKeyOnDemand"
      ],
      "Resource": "*"
    }
  ]
}
```

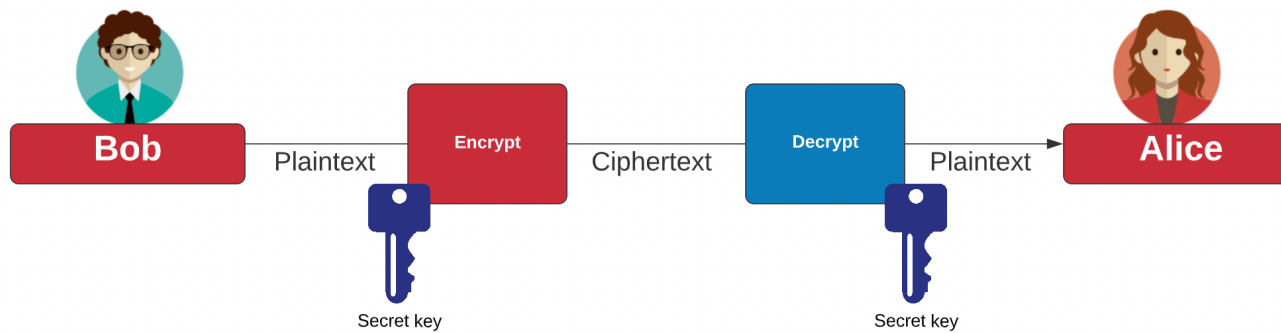
```

},
{
  "Sid": "Allow use of the key",
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::590269919252:role/voclabs"
  },
  "Action": [
    "kms:Encrypt",
    "kms:Decrypt",
    "kms:ReEncrypt*",
    "kms:GenerateDataKey*",
    "kms:DescribeKey"
  ],
  "Resource": "*"
},
{
  "Sid": "Allow attachment of persistent resources",
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::590269919252:role/voclabs"
  },
  "Action": [
    "kms:CreateGrant",
    "kms:ListGrants",
    "kms:RevokeGrant"
  ],
  "Resource": "*",
  "Condition": {
    "Bool": {
      "kms:GrantIsForAWSResource": "true"
    }
  }
}
]
}

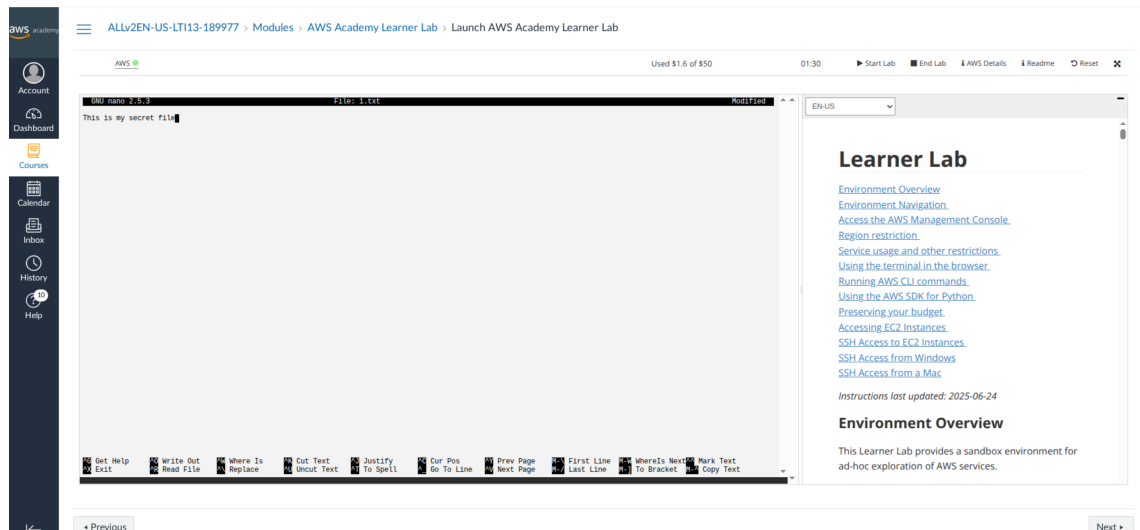
```

AWS and Symmetric Key

With symmetric key encryption, Bob and Alice use the same encryption key to encrypt and decrypt. In the following case, Bob and Alice share the same encryption key, and where Bob encrypts plaintext to produce ciphertext. Alice then decrypts with the same key, in order to recover the plaintext:



In the Amazon Learners Lab CLI, now we can create a file named 1.txt, and enter some text:



Once we have this, we can then encrypt the file using the “aws kms encrypt” command, and then use “fileb://1.txt” to refer to the file:

```
aws kms encrypt --key-id alias/MySymKey --plaintext fileb://1.txt --query CiphertextBlob --output text > 1.out
cat 1.out
```

This produces a ciphertext blob, and which is in Base64 format:

```
AQICAHgTBDpVTrBTrduWKdNnvMoMMUWjObqp+GqbghUx7qa6JwEQ7F2Fzubd+pcz3I06bFuLAAAdjB0BgkqhkiG9w0BBwagZzBlAgEAMGAGCSqGSib3
DQEHATAeBgllghkgBZQMEAS4wEQQMgl3vWRVPyL7KK3klAgEQgDP+dQ4KsqT94hiARF8zlybFAtXJJBIucc8M952KHmkJzBGQQP4f8YQQ70DELv97ZXiz
zME=
```

We could transmit this in Base64 format, but we need to convert it into a binary format for us to now decrypt it. For this we use the “Base64 -d” command:

```
base64 -i 1.out --decode > 1.enc
cat 1.enc
```

The result is a binary output:

```
eee_W_6848988@runweb254527:~$ cat 1.enc
00d0_a_s`He.0b-{su0s      *H
      *pVc3_]2H7N=u|]w=E#P2+&Teee_W_6848988@runweb254527:~$
```

Now we can decrypt this with our key, and using the command of:

```
aws kms decrypt --key-id alias/MySymKey --output text --query Plaintext --ciphertext-blob fileb://1.enc > 2.out
cat 2.out
```

The output of this is our secret message in Base64 format:

```
VGhpcyBpcyBteSBzZWNyZXQgZmlsZS4K
```

and now we can decode this into plaintext:

```
base64 -i 2.out --decode
eee_W_6848988@runweb254527:~$ aws kms decrypt --key-id alias/MySymKey --output text --query Plaintext --ciphertext-blob fileb://1.enc > 2.out
eee_W_6848988@runweb254527:~$ cat 2.out
VGhpcyBpcyBteSBzZWNyZXQgZmlsZQo=
eee_W_6848988@runweb254527:~$ base64 -i 2.out --decode
This is my secret file
eee_W_6848988@runweb254527:~$
```

The commands we have used are:

```
aws kms encrypt --key-id alias/MySymKey
cat 1.out
echo "== Ciphertext (Binary)"
base64 -i 1.out --decode > 1.enc
cat 1.enc
aws kms decrypt --key-id alias/MySymKey --output text --query Plaintext --ciphertext-blob fileb://1.enc > 2.out
```

```
echo "== Plaintext (Base64) "  
cat 2.out  
echo "== Plaintext"  
base64 -i 2.out --decode
```

and the result of this is:

```
== Ciphertext (Base64)  
AQICAHgTBDpVTrBTrduWKdNnvMoMMUWjObqp+GqbghUx7qa6JwEfz+s9z3e0Mw0tOzuB5LuYAAAAAdjB0BgkqhkiG9w0BBwagZzBlAgEAMGAGCSqGS Ib3  
DQEHATAeBg lghkgBZQMEAS4wEQQMqqwXsxB5QlQGVqZWAgEQgDOyBv6KYg4wN2bU/ZKSJ+5opJXMrjQj9GGvuuD2/Jeto9Er5yS91/iCb896CzCSeqUY  
Jeo=  
  
== Ciphertext (Binary)  
x:UNS)g  
00e0`v0t`He.0'=w*H  
yBTVV3b07f'h4#a+$oz  
0z%  
  
== Plaintext (Base64)  
VGhpcyBpcyBteSBzZWNYZXQgZmlsZS4K  
  
== Plaintext  
This is my secret file.
```

Here's a sample run in an AWS Foundation Lab environment:

```

ddd_v1_w_W3n_1455598@runweb67940:~$ cat 1.txt
This is my secret file.
ddd_v1_w_W3n_1455598@runweb67940:~$ cat go1.bat
aws kms encrypt --key-id alias/BillsNewKey --plaintext file://1.txt --query CiphertextBlob --output text > 1.out
echo "Ciphertext (Base64)"
cat 1.out
echo "Ciphertext (Binary)"
base64 -i 1.out --decode > 1.enc
cat 1.enc
aws kms decrypt --key-id alias/BillsNewKey --output text --query Plaintext --ciphertext-blob file://1.enc > 2.out
echo "Plaintext (Base64)"
cat 2.out
echo "Plaintext"
base64 -i 2.out --decode
ddd_v1_w_W3n_1455598@runweb67940:~$ ./go1.bat
bash: ./go1.bat: No such file or directory
ddd_v1_w_W3n_1455598@runweb67940:~$ ./go1.bat
Ciphertext (Base64)
AQICAHgTBDpVTtBTduWkdNnVwMmUWj0bqp+GqBghUx7qa6JwFSbOd+JF/Vw0Q4Gti65qIXAAAAdjB0B8gkqhkiG9w0BB8wagZzB1AgEAMGAGCSqGSIb3DQEHATAe
Bg1ghkgB8ZQMES4wEQQMAYIfakWswCQVwHmdAgEQgDNIeSdK3yYw0xkS2I9gFFSPappWeHuTV+6dptX+p0HVD4wkqaXpxUSoqhr+8uP7X2PWTrI=
Ciphertext (Binary)
x:UNSjg
00e0 1'He.0'R1~$D8:v0t *H
JE$s3Hy'J'&`|TjVx(WMA$D_cNPlaintext (Base64)
VGhpCy8pcy8teSBzZWMyZXQgZmlsZS4K
Plaintext
This is my secret file.
ddd_v1_w_W3n_1455598@runweb67940:~$ █

```

EN-US

Learner Lab Foundation Level

[Environment Overview](#)
[Environment Navigation](#)
[Access the AWS Manage](#)
[Region restriction](#)
[Service usage and other re](#)
[Using the terminal in the br](#)
[Running AWS CLI commanc](#)
[Using the AWS SDK for Pyt](#)
[Preserving your budget](#)
[Accessing EC2 Instance\(s\).](#)

Using Python

Along with using the CLI, we can create the encryption using Python. In the following, we use the boto3 library, and have a key ID of “MySymKey” and which is in the US-East-1 region:

First, create the python file:

```
nano kms_encrypt.py
```

And in it paste in this code

```

import base64
import boto3
from botocore.exceptions import ClientError

```

```
AWS_REGION = 'us-east-1'
```

```

KEY_ALIAS = 'alias/MySymKey'

kms_client = boto3.client("kms", region_name=AWS_REGION)

def encrypt(secret, key_id):
    try:
        response = kms_client.encrypt(
            KeyId=key_id,
            Plaintext=bytes(secret, encoding='utf8'),
        )
    except ClientError:
        print('Problem with encryption.')
        raise
    else:
        return base64.b64encode(response["CiphertextBlob"])

def decrypt(ciphertext, key_id):
    try:
        response = kms_client.decrypt(
            KeyId=key_id,
            CiphertextBlob=bytes(base64.b64decode(ciphertext)),
        )
    except ClientError:
        print('Problem with decryption.')
        raise
    else:
        return response['Plaintext']

print(f'Using KMS key: {KEY_ALIAS}')

msg = 'Hello'
print(f"Plaintext: {msg}")

cipher = encrypt(msg, KEY_ALIAS)
print(f"Cipher: {cipher}")

```



```
plaintext = decrypt(cipher, KEY_ALIAS)
print(f"Plain: {plaintext.decode()}")
```

Then run it

```
python3 kms_encrypt.py
```

Each of the steps is similar to our CLI approach. A sample run gives:

```
eee_w_6848988@runweb254527:~$ python3 kms_encrypt.py
/usr/lib/python3.7/site-packages/boto3/compat.py:82: PythonDeprecationWarning: Boto3 will no longer support Python 3.7 starting December 13, 2023. To continue receiving service updates, bug fixes, and security updates please upgrade to Python 3.8 or later. More information can be found here: https://aws.amazon.com/blogs/development/python-support-policy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
Using KMS key: alias/MySymKey
Plaintext: Hello
Cipher: b'AQICAHgB0330yj0Ww1inYZoSyH/8Xw05/xSesB9zQvI3HqfX/AFfo5+tIFoXUVLyLvML26VFAAAAYzBhBgkqhkiG9w0BBwagVDBSAgEAME0GCSqGSIB3DQEHATAeBgLghkgBZQMEAS4wEQQMwbtlw4wbo6t9nKS3AgEQgCBtKvixZk0um63qoXAFftaY2lWJcdRphsU1kPc8yzQt2A=='
Plain: Hello
eee_w_6848988@runweb254527:~$
```